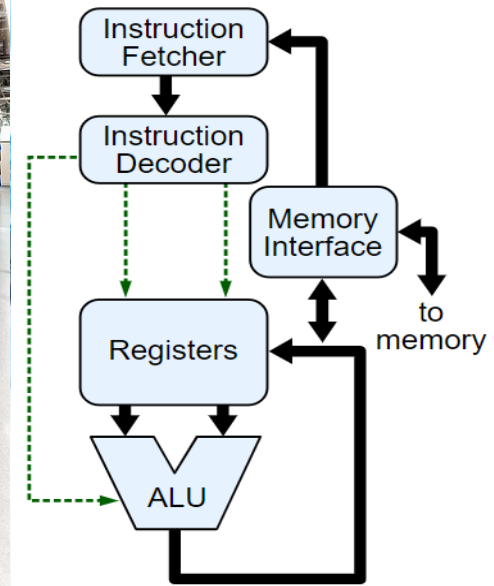
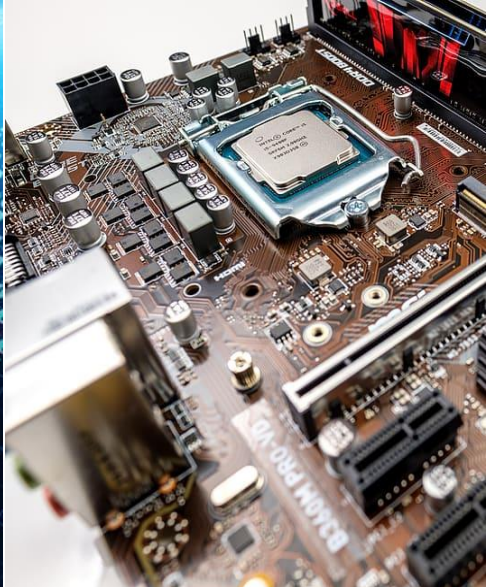




Pulse survey

- ▶ Collected 8 responses, thank you!
 - The pre-recorded lectures are not available soon enough (3/8).
 - CWs are short and non engaging. (2/8)

Recap

- ▶ We have now looked at all the key R-format and I-format MIPS instructions at a low level:
 - Their representation in terms of 32-bit machine code words.
 - How the machine executes them in terms of manipulating bits and bytes in registers and memory.

Overview

- ▶ In this lecture we will examine how high-level language constructs are converted into MIPS assembly.
- ▶ We will also cover the final instruction format, J-type, used for several purposes such as function calling.
- ▶ Background Reading:
Patterson & Hennessey, Chapter 2.

Starting to program MIPS using named registers “Register Usage Convention”

Name	Register Number	Usage
\$zero	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	function return values
\$a0-\$a3	4-7	function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	function return address

How MIPS handles arrays

High-level Code

```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

MIPS assembly code

```
# array base address = $s0 (say array stored at 0x12348000)  
lui    $s0, 0x1234          # put 0x1234 in upper half of $s0  
ori    $s0, $s0, 0x8000     # put 0x8000 in lower half of $s0  
  
lw     $t1, 0($s0)          # $t1 = array[0]  
sll    $t1, $t1, 1          # $t1 = $t1 * 2  
sw     $t1, 0($s0)          # array[0] = $t1  
  
lw     $t1, 4($s0)          # $t1 = array[1]  
sll    $t1, $t1, 1          # $t1 = $t1 * 2  
sw     $t1, 4($s0)          # array[1] = $t1
```

J-ump instructions



```
li $8, 10
j target
addi $8, $8 1
addi $8, $8 1
target:
```

► But addresses are 32-bits long!

– This limits the addresses we can jump to.



Address	Code	Basic
0x00400000	0x2408000a	addiu \$8,\$0,0x0000000a
0x00400004	0x08100004	j 0x00400010
0x00400008	0x21080001	addi \$8,\$8,0x00000001
0x0040000c	0x21080001	addi \$8,\$8,0x00000001

$$\text{NewPC} = (\text{PC}+4)[31:28] \cdot JA \cdot 00 = 4 \cdot JA = 0x400010$$

$$\Rightarrow JA = \frac{0x400010}{4} = 0x100004$$

Jump j J PC=JumpAddr (5) 2_{hex}

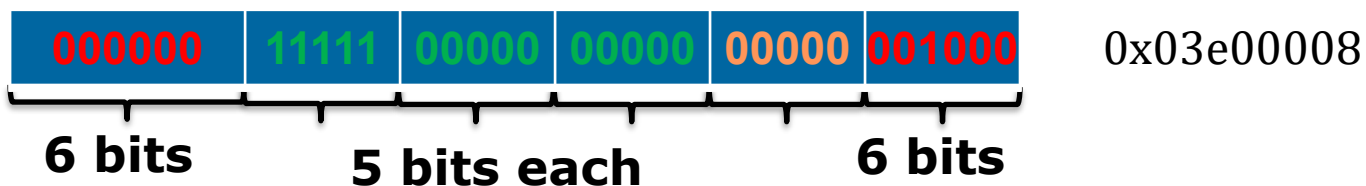
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }

J-ump instructions

- ▶ The most significant 4 bits of the PC are *copied on* J-ump instructions.
 - What should we do if we want to branch into a code region with different MSBs or is only known at runtime?

Jump Register jr R PC=R[rs] 0 / 08_{hex}

- An R-type instruction!
- jr \$ra #Remember: \$ra is the *name* of \$31



If ... else ... branching

High-level code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j

    bne $s3, $s4, L1
    addu $s0, $s1, $s2
    j    done
L1:   subu $s0, $s0, $s3
done:
```

While loops

High-level code

```
// determines the power  
// of x such that  $2^x = 128$ 
```

```
int pow = 1;  
int x   = 0;  
  
while (pow != 128) {  
    pow = pow * 2;  
    x = x + 1;  
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x  
  
addi $s0, $0, 1  
add  $s1, $0, $0  
addi $t0, $0, 128  
while: beq  $s0, $t0, done  
      sll  $s0, $s0, 1  
      addi $s1, $s1, 1  
      j    while  
  
done:
```

For loops

High-level code

```
// add the numbers
// from 0 to 9

int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
    sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum

        addi $s1, $0, 0
        add  $s0, $0, $0
        addi $t0, $0, 10
for:     beq  $s0, $t0, done
        add  $s1, $s1, $s0
        addi $s0, $s0, 1
        j    for
done:
```

Writing functions

- This is slightly more involved than the other high-level programming constructs discussed previously.
- This will lead us to a thorough discussion of the 'Stack Segment' of MIPS and register usage conventions.
- E.g., how can functions know not to overwrite each other's registers?

Functions are important abstractions in high level languages

A high level language function or function to calculate the absolute difference between two numbers:

```
int abs_diff(int a, int b) {  
    int result = (a - b);  
    if (result < 0) {  
        result = (b - a);  
    }  
    return result;  
}
```

We usually simply assume the computer knows how to call a function and generate an output.

But how does a processor call a function?

Calling functions – the `jal` and `jr` instructions

Some lines of program

0x0041AB28 _____
0x0041AB30 _____
0x0041AB34 _____
0x0041AB38 _____
0x0041AB3C **jal** "0x00445678" # \$ra ← PC + 4, PC ← operand
0x0041AB40 _____

Function call

**Function
Return**

Function Code

0x00445678 _____
0x0044567C _____
0x00445680 _____
0x00445684 _____
0x00445688 **jr** \$ra # \$ra → PC

i.e., 0x0041AB40 → \$ra
0x00445678 → PC

There is also a
`jalr` instruction

What does `jalr $s0` do?

Using saved registers for local variables

High-level code

```
// Integers represented  
// as 32-bit signed words.
```

```
int a = 64;
```

```
int b = 305419896;
```

```
func()
```

```
int c = a + b;
```

MIPS assembly code

```
# $s0 = a  
addi $s0, $0, 0x0040  
  
# $s1 = b  
lui $s1, 0x1234  
ori $s1, $s1, 0x5678  
  
# Or pseudo-instruction  
# li $s1, 0x12345678  
  
jal func  
  
# $s2 = c  
add $s2, $s0, $s1
```

Functions are important abstractions in high level languages

A high level language function or function to calculate the absolute difference between two numbers:

```
int abs_diff(int a, int b) {
```

```
    int result = (a - b);
```

```
    if (result < 0) {
```

```
        result = (b - a);
```

```
    }
```

```
    return result;
```

```
}
```

So we use `ja1` to call the function and `jr` to return.

We use saved registers to stuff we need later.

What else?

Things to think about

1. Where to the input variables for the function?
2. Where to store the 'result' variable within the function?
3. Where to put the output for the caller?

Registers and Usage Conventions

Register Name	Register Number	Usage
\$v0	\$2	Result value
\$v1	\$3	Result value (if 64-bit result or two 32-bit results are generated)
\$a0	\$4	Argument 1.
\$a1	\$5	Argument 2.
\$a2	\$6	Argument 3.
\$a3	\$7	Argument 4.
\$ra	\$31	Return address.

Registers and Usage Conventions

Register Name	Register Number	Usage
\$t0, \$t1, \$t2, \$t3, \$t4, \$t5, \$t6, \$t7	\$8 to \$15	Temporary registers. <i>Do not need to be preserved</i> across function calls
\$s0, \$s1, \$s2, \$s3, \$s4, \$s5, \$s6, \$s7	\$16 to \$23	Saved registers. The function <i>must</i> save and restore these if it wants to use them.
\$t8, \$t9	\$24, \$25	More temporary registers.

Creating a function for calculating the absolute value

```
# abs_diff(a, b) calculates the
# absolute difference of two integers.

abs_diff:
    sub $v0, $a0, $a1    # result = a - b
    bgez $v0, return     # If +ve jump next instruction.
    sub $v0, $a1, $a0    # result = b - a

return:
    jr $ra               # Return from function.
```

- Reuse of 'abs_diff' function within your code (abstraction)
- Cleaner more compact code.
- Creation of library function/functions (i.e. sqrt, sin, etc.)

Why ?

Putting it together

```
# Calculating absolute difference
```

```
.data
```

```
vals: .word 180, 100, 0
```

```
.text
```

```
.globl main
```

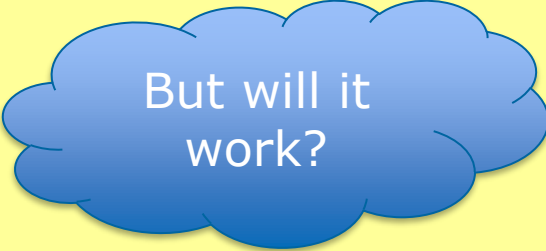
```
main: jal my_program      # Run my program.
      li $v0, 10           # Exit program using system call 10.
      syscall             # syscall 10 (exit)
```

```
my_program:
```

```
    la $t1, vals          # Load address of data into $t1.
    lw $a0, 0($t1)         # Load A from memory into $a0.
    lw $a1, 4($t1)         # Load B from memory into $a1.

    jal abs_diff           # Jump and link to 'abs_diff' function.

    sw $v0, 8($t1)         # Write result and return.
    jr $ra                # Return to main program.
```



But will it
work?

What is wrong with our code?

```
# Calculating absolute difference
.data
vals: .word 180, 100, 0

.text
.globl main

main: jal my_program      # Run my program.
      li $v0 10           # Exit program using system call 10.
      syscall            # syscall 10 (exit)

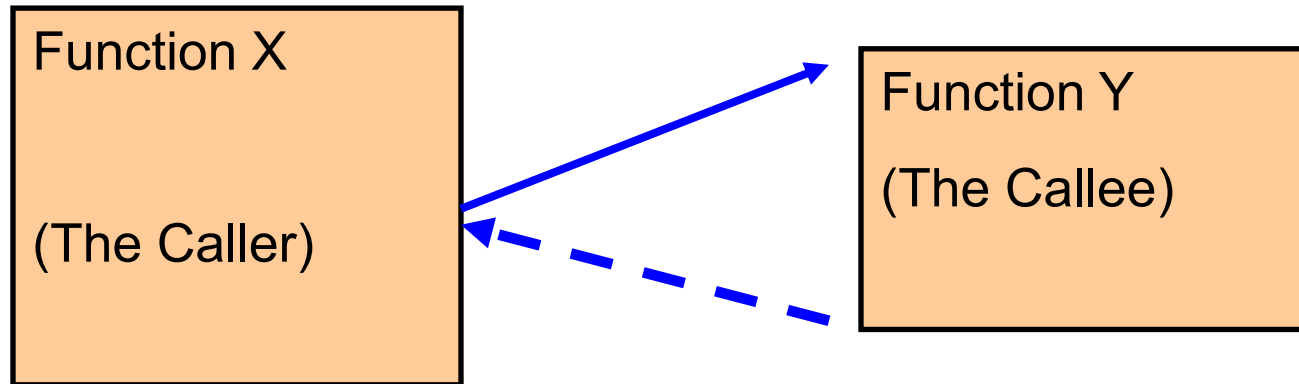
my_program:
      la $t1, vals        # Load address of data into $t1.
      lw $a0, 0($t1)      # Load A from memory into $a0.
      lw $a1, 4($t1)      # Load B from memory into $a1.

      jal abs_diff        # Jump and link to 'abs_diff' function.

      sw $v0, 8($t1)      # Write result and return.
      jr $ra              # Return to main program.
```



Functions need not to interfere with each other's registers



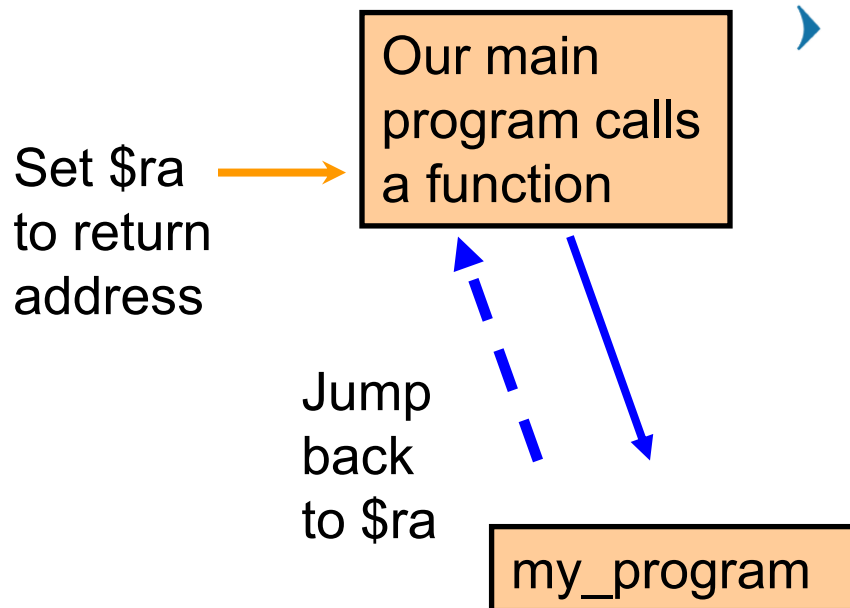
If we don't want a register modified, then either:

- ▶ The *caller* needs to save the register value before making the function call.

Or

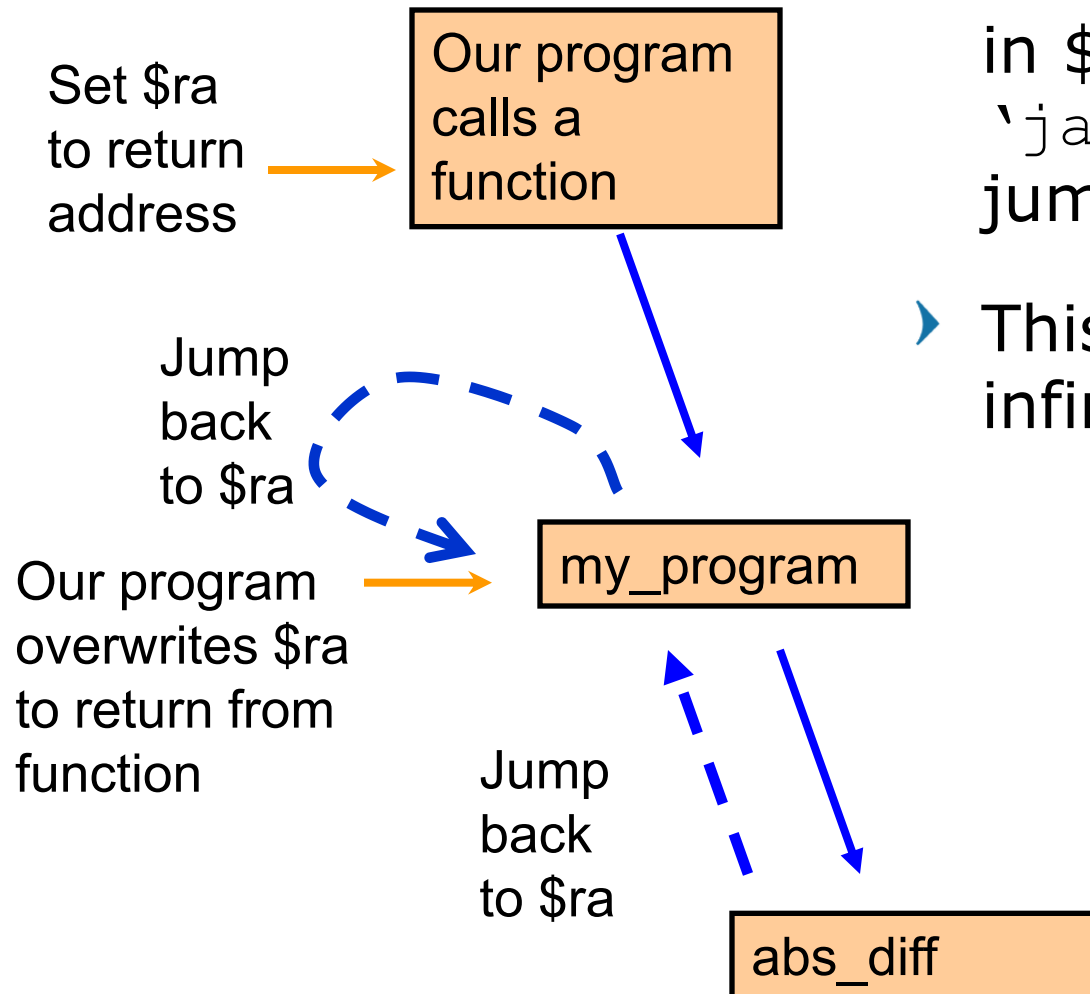
- ▶ the *callee* needs to save the register before it modifies it and restore it before returning.

When one function calls another



- Remember: `jal` sets `$ra` so the function can return to our main program.

Problems after introducing `abs_diff`



- ▶ We overwrite the value in `$ra` when we do a `'jal'` operation to jump to our `abs_diff`.
- ▶ This results in an infinite loop.

But how do we 'save' and 'restore' registers?

- ▶ We cannot really save them in 'other registers' since we do not know if these will also be used elsewhere.
- ▶ We must save them to memory
 - But we cannot save a register (say, \$s0) into the same memory location (e.g., 0x10010100) each time, since nested calling of functions will overwrite this saved value with another value.
- ▶ This is where the 'Stack' is particularly useful.

Other special named registers

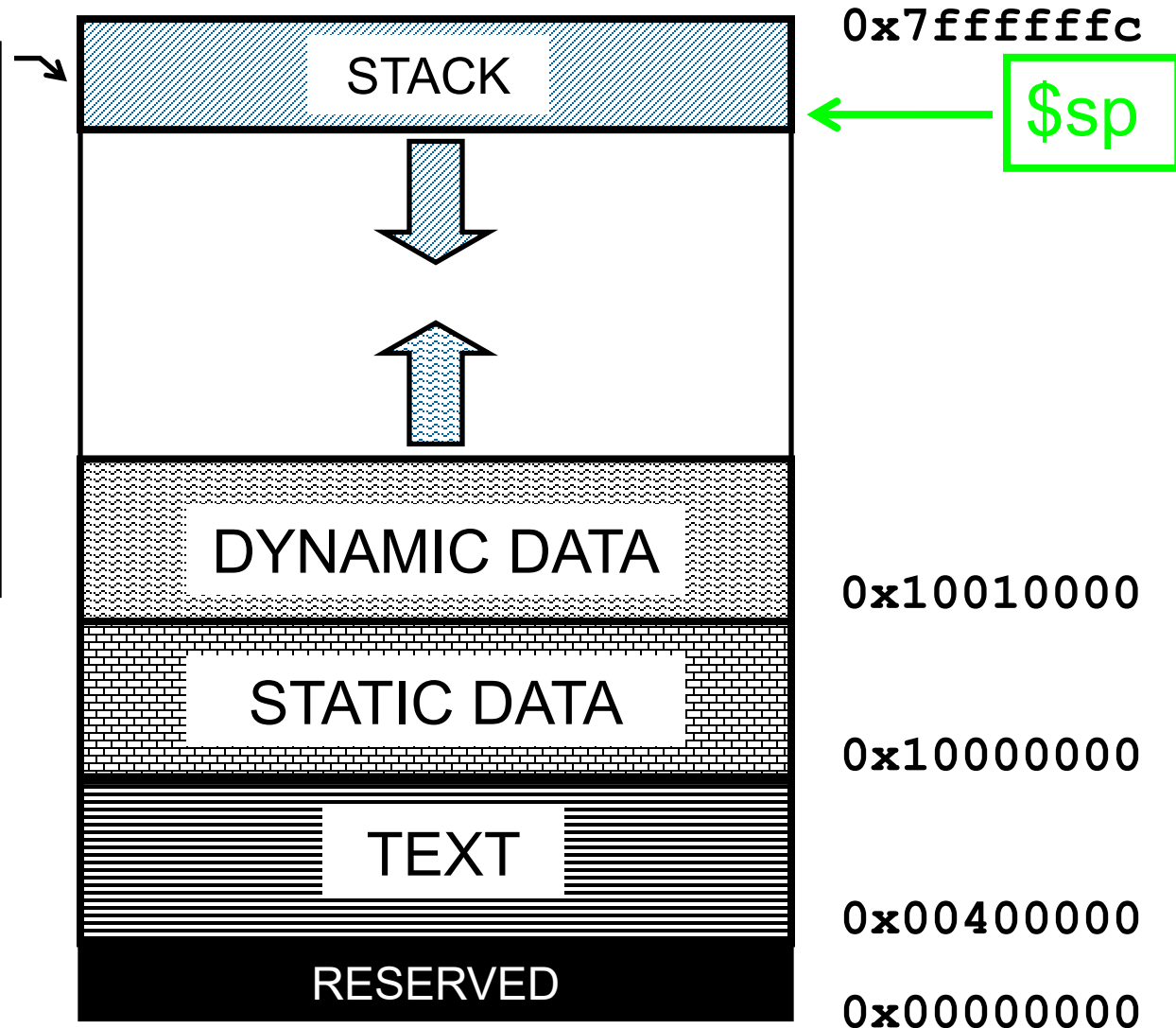
Register Name	Register Number	Usage
\$at	\$1	Reserved for the assembler for implementing pseudo instructions.
\$k0, \$k1	\$26, \$27	Reserved for operating system kernel
\$gp	\$28	Useful pointer into the global static data segment (set to 0x10008000).
\$sp	\$29	Stack Pointer – more about this soon
\$fp	\$30	Frame Pointer – more about this soon
\$ra	\$31	Return address from function

See the MIPS Card: \$gp, \$sp, \$fp, and \$ra also need to be saved and restored if changed by a function.

Recall the Stack memory segment

The stack is for storing information that needs to be remembered within a function.

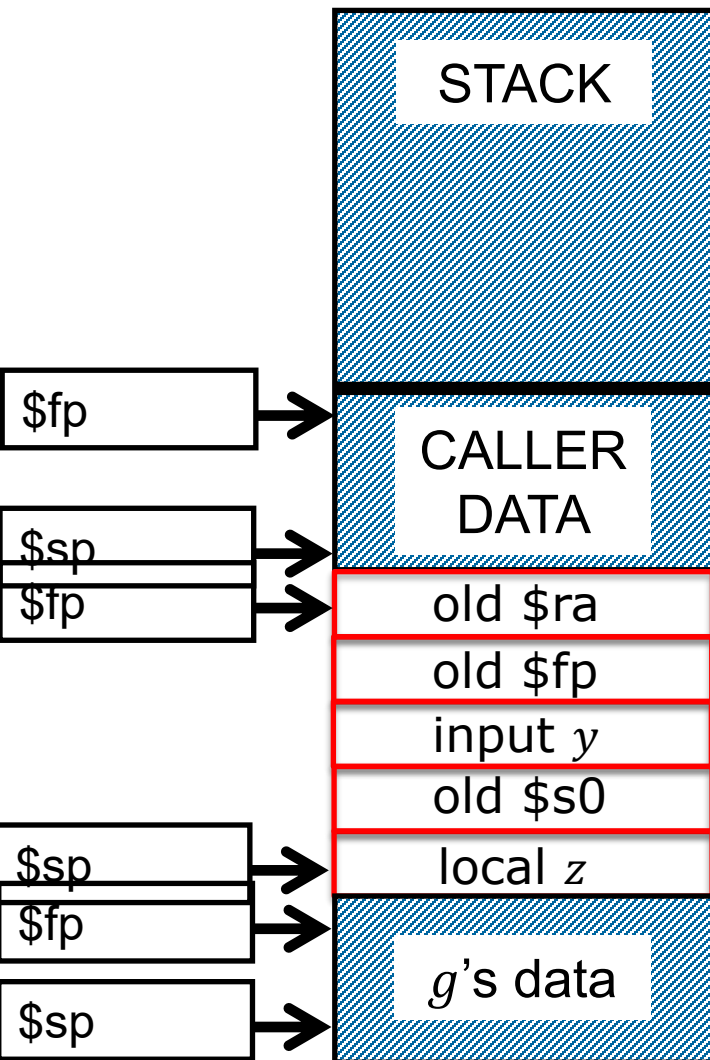
The stack pointer register (\$sp) points to the end of the stack.



What's in the stack?

A typical one-argument function

```
int f(int y)
```



```
addiu $sp, $sp, -20
sw $ra, 16($sp)
sw $fp, 12($sp)
sw $a0, 8($sp)
sw $s0, 4($sp)
addiu $fp, $sp, 16
```

Some code that:

- *uses \$s0 and y*
- *stores a local variable z #in -16(\$fp)*
- *calls a function g(2y+z) # g terminates*
- *continues to use \$s0 uninterrupted!*
- *still has access to y (-8(\$fp), not \$a0) and z*

```
lw $ra, 16($sp)
lw $fp, 12($sp)
lw $s0, 4($sp)
addiu, $sp, $sp, 20
```

Does it work now ? It should ...

```
# Calculating absolute difference without infinite loop !
my_program:
    addi $sp, $sp, -8 # Move the stack pointer.
    sw $ra, 4($sp)    # Store return address.
    sw $fp, 0($sp)    # Store frame pointer.
    addi $fp, $sp, 4  # Set new frame pointer.

    la $t1, vals      # Load address of data into $t1.
    lw $a0, 0($t1)     # Load A from memory into $a0.
    lw $a1, 4($t1)     # Load B from memory into $a1.
    jal abs_diff       # Jump and link to 'abs_diff' function.
    la $t1, vals       # Load address of data into $t1 again.

    sw $v0, 8($t1)     # Write result and return.
    lw $ra, 4($sp)     # Restore return address.
    lw $fp, 0($sp)     # Restore frame pointer.
    addi $sp, $sp, 8   # Restore stack pointer position.

    jr $ra             # Return from program.
```

Summary

- ▶ We saw how high-level language constructs are coded in assembly language.
- ▶ Looping constructs and function calls are important abstractions for high-level languages so it's good to understand how they work at the machine level.
- ▶ This brings out a number of important themes such as register conventions and stack usage.
- ▶ Next week we will examine how to call 'operating system services' and discuss floating point operations.